

An Adaptive IDS Scheduling Framework for Host Based Mimicry Attack Detection in IoT Networks

Surja Sanyal, Manas Khatua

Department of Computer Science and Engineering
Indian Institute of Technology Guwahati
Guwahati, India
{surja.sanyal, manaskhatua}@iitg.ac.in

Pratik Chattopadhyay

Department of Computer Science and Engineering
Indian Institute of Technology (BHU) Varanasi
Varanasi, India
pratik.cse@iitbhu.ac.in

Abstract—With the expanding applicability of the Internet of Things (IoT), novel IoT network security challenges also appear more frequently. Host-based Mimicry Attacks (HMA) are one of them that is difficult to detect by traditional Intrusion Detection Systems (IDS) since they mimic legitimate network and host behaviors to evade detection. Additionally, IDSs consume host device resources when their implementations, partially or fully, are executed in the device, and thus it is needed to be scheduled properly so as to save device resources when possible. Current IDS solutions capable of detecting HMAs mostly use random-based scheduling or do not use scheduling at all and so are always executed. Random-based scheduling solutions have low HMA detection accuracy, while solutions without scheduling do not save device resources. This work proposes an adaptive IDS scheduling framework that suspends IDS execution based on device trust and resumes IDS execution when the device is under attack. Extensive experimentation using the Cooja simulator and Python script shows that the proposed framework achieves a device behavior classification accuracy of at least 97.15% when under attack while saving at least 81.98% of CPU energy when not under attack for a network of 40 devices, as compared to those when executing the IDS without any scheduling scheme.

Index Terms—Internet of Things (IoT), network security, Intrusion Detection System (IDS), mimicry attack, resource-saving, IDS scheduling

I. INTRODUCTION

The Internet of Things (IoT) finds use in various fields of application, including those that require high network security against adversarial attacks [1]. Advanced attacks, such as Host-based Mimicry Attacks (HMA), mimic legitimate host and network activity to evade detection by Intrusion Detection Systems (IDS) while carrying out malicious activities affecting the host [2]. IDSs can be signature/rule-based, which checks for known attack footprints [3]; or anomaly-based, which checks for unknown patterns [4]; or hybrid, which combines both approaches [5]. Often anomaly-based IDSs use machine/deep learning (ML/DL) methods to increase their productivity [6]. IDSs can also be classified as network activity-based (NIDS) [7], host behavior-based (HIDS) [8], or hybrid IDS using both approaches [9]. However, device resources are consumed whenever a portion of the IDS implementation executes in a host device. To this end, better feature engineering for efficient learning [10], [11], dimensionality reduction for lower computation load [12], external sensors and IDS hardware for workload delegation [8], resource-rich cloud, fog, and gateway

routers for computation offloading [9], [13], smaller and quantized detection models for lower resource consumption [7], shallow ensemble detectors and statistical models for lesser computation demands [14], and lightweight DL model layers for lighter models [8] have been proposed. However, executing an IDS in devices not under attack unnecessarily depletes its resources. To reduce resource consumption, the execution of the IDS can be suspended when not required. Most solutions that detect HMAs use random-based IDS execution schedule [15] or do not use any scheduling scheme [7]. Random-based solutions have low HMA detection accuracy as they do not use attack footprints in the scheduling of the IDS, while IDSs without any scheduling do not save device resources.

This work proposes an IDS scheduling framework that addresses the aforementioned issues. The proposed framework uses a monitor module that tracks the present state of a device using the traffic load parameter and the attack confidence score generated by the execution of the IDS. The monitor module is supplemented by a threshold generator which provides the cutoff values of the parameters dynamically, a backoff scheme that suspends IDS execution based on trust, an aging scheme that revives the IDS from suspension, and an execution scheme that adapts to diverse objectives under different device states. To summarize, the contributions of this article are as follows:

- An IDS execution scheme is proposed that switches between the primary scheduling objectives of resource-saving and detection accuracy based on the device state.
- A runtime device monitoring scheme that tracks traffic load changes and IDS attack confidence parameters to estimate the current state of the host device.
- A dynamic threshold generation scheme for the monitored parameters that updates as per the attack situation.
- An adaptive backoff scheme to encourage energy saving during infrequent device attack conditions.
- A nonlinear aging scheme to ensure high detection accuracy in case of aperiodic device attack.

The remainder of this article is organized as follows. Section II summarizes the existing works in scheduling HIDSs in host devices. Section III presents the proposed HIDS execution scheduling solution. Section IV presents the evaluation results of the proposed solution. Section V presents the conclusion.

II. RELATED WORK

The application of IoT in various fields is expanding with time. With this expansion, the available attack surface is also increasing [1]. HMAs mimic legitimate host behavior to evade IDSs while carrying out malicious activities affecting the host [2]. While anomaly-based IDSs check for unknown patterns in host behavior and are useful in the detection of unknown attacks [7], signature/rule-based IDSs check for known malicious patterns and result in high detection rates of known attacks [3]. Hybrid IDSs combine both these approaches for the detection of both known and unknown attacks [5]. ML/DL-based methods produce better detection results for anomaly-based IDSs [6]. Network-based IDSs (NIDS) focus on the network behavior of the devices for the detection of attacks [4]. Host-based IDSs (HIDS) focus on analyzing the host behavior to check for probable attacks [8]. Hybrid IDSs use both these concepts for attack detection [9]. Anomaly-based IDSs with their component running in the hosts detect HMAs.

IDSs executing in a host device consume resources of the device [7]. To reduce this resource consumption, several approaches have been taken. Selection of proper features to represent device behavior requires less computations [10], [11]. Improved learning methods with multimodal approaches towards data dimension reduction consume fewer resources [12]. External hardware and compute capabilities can be leveraged to increase network lifetime [8]. Utilization of resource-rich cloud, fog, and gateway routers for compute-intensive tasks is another useful approach [9], [13]. Smaller and more quantized attack detection models reduce the compute requirements of the IDS [7]. Architectural changes such as shallow ensemble models or lightweight DL model layers demand less resource [14]. Statistical methods instead of DL-based ones can further cut down the resource demands [8]. Each execution of an IDS consumes resources, however minimized. Therefore, the execution of an IDS can be scheduled in a host device to save device resources whenever possible. Several solutions have been followed to this end. Random-based IDS execution scheduling has been proposed [15]. Placement optimization of the IDS has been modeled as a form of vertex cover problem by centralized/hybrid IDS implementations [16]. Cluster head-based approaches have been proposed for IDS placement and execution [17]. Iterative stochastic game theoretic models have been used by centralized/hybrid IDS implementations to predict devices under attack [18]. Using ML-based methods to solve mathematical IDS placement models has also been proposed [19]. Flow-based and graph-based network traffic behaviors have been utilized as well [20].

Most of the existing scheduling schemes do not detect HMAs. A majority of the aforementioned solutions use a centralized/hybrid approach for IDS placement, which incurs extra control communication overhead. Randomized approaches have low HMA detection accuracy. Game theoretic approaches assume the values of several attacker parameters which are often unknown for the IDS. ML-based and multimodal flow-based solutions incur extra computation or communication

TABLE I
EXISTING WORKS OF IDS SCHEDULING FOR HMA DETECTION IN IOT.

Scheme	HDA	RC	UP	CC	CN	Scheduling
[7]–[9]	Low	High	×	✓	✓	×
[15]	Medium	Low	×	×	×	Random
[16]	×	Medium	×	×	✓	Graph
[17]	×	High	×	✓	✓	Cluster
[18]	×	Medium	✓	×	✓	Game
[19]	×	High	×	×	✓	ML
[20]	×	High	×	✓	✓	Graph, Flow
This work	High	Low	×	×	×	Traffic load

HDA = HMA Detection Accuracy, RC = Resource Consumption,

UP = Unknown Parameters, CC = Computation overhead on Cluster,

CN = Communication overhead on Network, ✓ = Present, × = Absent.

overheads. In brief, all the aforementioned solutions have at least one issue among low HMA detection accuracy, unknown adversary parameters, extra communication costs, and device resource consumption. The summary of the notable features of the existing works is presented in Table I.

III. PROPOSED METHODOLOGY

Periodic execution of any IDS is performed at the expiry of each time period (hereafter, cycle) in order to detect any attacks, including HMAs, on the host device during that cycle. This execution at each cycle consumes device resources and depletes the host devices and the network. This work proposes an IDS scheduling framework to address this challenge. The proposed solution consists of the threshold generator (section III-C), the backoff module (section III-D), the aging module (section III-E), the adaptive execution scheme (section III-F) and the monitor module (section III-G). The attack model and the HIDS are discussed in section III-A and section III-B, respectively. Fig. 1 presents the modular interactions of the proposed solution. Host device behavior is captured in the *packet transmission* (Tx) and *CPU computation* energies consumed by the device. These features are used by the IDS as inputs to generate a *confidence score* indicating the confidence of an ongoing attack. This *confidence score* and the *total transmission energy* are fed to the monitor module for device state estimation. The dynamic threshold generator generates dynamic threshold values that help the monitor module adapt to the variable HMA rate. The backoff module provides adaptive backoff spaces to the monitor module to help it suspend the execution of the IDS based on device trust. The aging module provides nonlinear aging values to the monitor module to revive the IDS execution from suspension. The IDS execution scheme decides the primary and secondary objectives of the scheduling according to the device state. These components are discussed in detail next.

A. Host-oriented Mimicry Attack (HMA) Model

HMAs are advanced attacks that target host devices while mimicking legitimate behavior to evade detection by IDSs [2]. In this work, the attacker is assumed to be resident in one or more network devices. The attacks are designed by inserting attack code pieces at randomly chosen locations in benign device behavior sequences [21]. The attacks aim to alter the

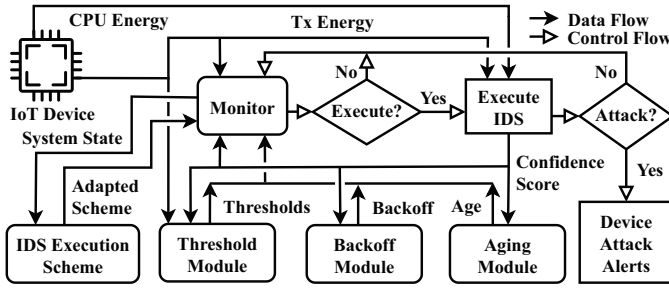


Fig. 1. Modular interactions of the proposed solution residing in a host device.

Algorithm 1: HIDS Training and Deployment.

Input: T_{atk}, X, Y^t

Output: C^t

```

1 create autoencoder  $AE$  and train using benign data  $X$ 
2  $R_{th} \leftarrow$  maximum reconstruction error of  $AE(X)$ 
3 foreach cycle  $t$  do
4   if IDS execution is scheduled then
5      $R^t \leftarrow$  Reconstruction error vector  $AE(Y^t)$ 
6      $V^t \leftarrow$  count threshold violations  $R^t > R_{th}$ 
7      $C^t \leftarrow V^t / T_{atk}$  clipped at 1

```

radio duty cycle of the compromised devices so as to consume more power while not modifying system call sequences so as to evade detection. Eventually, this results in the depletion of the resources of the network. Also, HMA attack probabilities are dynamic over time and make it difficult for IDSs with an execution schedule to have high HMA detection accuracy.

B. Host Based Intrusion Detection System (HIDS) Model

The HIDS architecture is adapted from [7] to generate the ground truth. It is a long short-term memory (LSTM) [22] and multilayer perceptron (MLP) [23] based autoencoder (AE) [24], trained without supervision to reconstruct inputs. The input, the output, and the bottleneck layers of the AE have the dimensions of two, two, and one, respectively. Algorithm 1 presents the training and deployment of the HIDS model. It accepts as inputs the attack tolerance T_{atk} (i.e., the number of detected attack attempts after which an alert is raised), the benign host behavior vector X , and the deployment host behavior vector Y^t for cycle t . It outputs the current attack confidence score. An autoencoder AE is created and trained without supervision using X (shown in line 1). The maximum reconstruction error post-training acts as the reconstruction error threshold R_{th} during deployment (shown in line 2). The IDS executes if scheduled by the proposed framework for that cycle. The input reconstruction error vector R^t from the AE is obtained using the deployment host behavior vector Y^t (shown in line 5). The number of threshold violations is obtained as $V^t = \sum(R^t > R_{th})$ (shown in line 6). The attack confidence score is $C^t = \frac{V^t}{T_{atk}}$, clipped at 1 (shown in line 7).

C. Proposed Dynamic Threshold Generator

Dynamic threshold values of change in traffic load and confidence score are provided to the monitor module for state estimation. At the start of the deployment phase, both the thresholds for traffic and confidence are initiated at 0.5. These thresholds are updated each time the IDS executes and detects an attack. The thresholds are updated as the average of the current thresholds and 50% of the current changes. Thus, for traffic load E^t , updated threshold is $E_{th}^{t+1} = [E_{th}^t + E^t/2]/2$. For the confidence parameter, updated threshold is $C_{th}^{t+1} = [C_{th}^t + (C^t - C_{th}^t)/2]/2$. Only 50% of the changes are used in the update to dampen sharp parameter fluctuations.

D. Proposed Adaptive Backoff Scheme

The trust-based backoff scheme is initiated every time the IDS executes, and the device state S^t is estimated as *Benign*. To leverage the fact that a majority of the devices are benign and do not require repeated IDS executions, a trust-based backoff scheme is proposed. In this scheme, the backoff period increases with the number of past cycles p with IDS executions that resulted in the device state S^t being estimated as *Benign*, where $p \leq t$. To achieve the same, the backoff space is designed as $b^t = p^3$. Backoff resets to 1 at $p > 10$. Thus, the backoff space $b^t \in [1, 1000]$ increases with the number of past cycles p with IDS executions when not under attack.

E. Proposed Nonlinear Aging Scheme

HMAs aim to evade detection by interleaving attack attempts with benign behavior. Thus, many IDS execution cycles result in device state S^t being estimated as *Benign*. The proposed backoff scheme will be triggered to deter further IDS executions in order to save device resources. Attackers can leverage this situation by aperiodically attacking for several cycles before the backoff space expires. Confidence scores will not be available until the monitor module instructs the IDS to execute. To counter such attackers, a nonlinear aging scheme $a^t = \frac{m^2}{b^t}$ has been proposed that grows at a quadratic pace in the backoff space with the linear growth in the m number of contiguous cycles without IDS execution.

F. Proposed Flexible IDS Execution Scheme

Two primary objectives are taken into account for scheduling the IDS. The first one is high HMA detection accuracy during *Attack* state, and the second one is high energy saving during *Benign* state of the device. To achieve this adaptive switch between these two modes as required, the IDS is executed if at least one of the following conditions is fulfilled:

- Both traffic and confidence thresholds are violated: $E^t > E_{th}^t$ and $C^t > C_{th}^t$. This promotes energy saving by the execution scheme when the device state is *Benign*.
- The average of the two fractional violations of the traffic and the confidence thresholds, and a device trust weighted random choice, is above the attack probability threshold $P_{th}\%$: $[E^t/E_{th}^t + C^t/C_{th}^t + r^t]/3 > P_{th}\%$. This r^t is a random choice between 0 and 1 with current device trust $d^t = \frac{\sum_{i=0}^t (S^i \text{ is } \textit{Benign})}{(t+1)}$ favoring 0. This promotes high detection accuracy when the device state is *Attack*.

Algorithm 2: Proposed IDS Scheduling Framework.

Input: ε^t current device traffic vector**Output:** S^t current device state

```
1 Function Schedule ( $\varepsilon^t$ ):  
2    $m, p \leftarrow$  initialize aging and backoff counters to 1  
3    $E_{th}^0, C_{th}^0 \leftarrow$  initialize thresholds to 0.5  
4    $C^0 \leftarrow$  initial confidence score from HIDS  
5   call Update ( $E^0 \leftarrow 1, C^0, E_{th}^0, C_{th}^0$ )  
6   foreach cycle  $t$  do  
7      $E^t \leftarrow$  change in traffic load  $\frac{|\varepsilon^0 - \varepsilon^t|}{\varepsilon^0}$   
8      $b^t \leftarrow$  backoff space  $p^3$   
9      $a^t \leftarrow$  aging value  $\frac{m^2}{b^t}$   
10     $C^t \leftarrow$  aged confidence score  $a^t + C^{t-1}$   
11     $d^t \leftarrow$  device trust factor  $\frac{\sum_{i=0}^t (S^i == Benign)}{(t+1)}$   
12     $r^t \leftarrow$  device trust  $d^t$  weighted random (0,1)  
13    if [ $E^t > E_{th}^t$  and  $C^t > C_{th}^t$ ] or [average ( $E^t/E_{th}^t$  and  $C^t/C_{th}^t$  and  $r^t$ )  $> P_{th}\%$ ] then  
14       $C^t \leftarrow$  get confidence score from HIDS  
15    else  $m \leftarrow$  aging counter increment to  $m + 1$   
16    call Update ( $E^t, C^t, E_{th}^t, C_{th}^t$ )  
17 Function Update ( $E^t, C^t, E_{th}^t, C_{th}^t$ ):  
18   if  $C^t \geq 1$  then  
19      $S^t \leftarrow$  device state set to Attack  
20      $\Delta E^{t+1} \leftarrow$  50% of change in traffic load  
21      $\Delta C^{t+1} \leftarrow$  50% of change in confidence score  
22      $E_{th}^{t+1} \leftarrow$  average of  $E_{th}^t$  and  $\Delta E^{t+1}$   
23      $C_{th}^{t+1} \leftarrow$  average of  $C_{th}^t$  and  $\Delta C^{t+1}$   
24      $m \leftarrow$  aging counter reset to 1  
25   else  
26      $S^t \leftarrow$  device state set to Benign  
27      $p \leftarrow$  backoff counter increment to  $p + 1$   
28     if  $p > 10$  then  $p \leftarrow$  backoff counter reset to 1
```

G. Proposed Runtime Device Monitoring Scheme

Each host device in the network is monitored using two parameters. The first one is the total packet transmission energy used by the device during a cycle t given by $\varepsilon^t \propto \tau^t$, where τ^t is the traffic load, and is available before the execution of the IDS. The absolute change in the traffic load is $E^t = \frac{|\varepsilon^0 - \varepsilon^t|}{\varepsilon^0}$. The second monitored parameter, available after the execution of the IDS, is the confidence score $C^t \in [0, 1]$ generated by the IDS execution and is updated as $C^{t+1} = a^t + C^t$, where a^t is the aging value. A device trust-based random number r^t is used to make the IDS execution unpredictable by diverse HMAs. Using r^t and the thresholds provided by the dynamic threshold generator, E_{th}^t and C_{th}^t , the monitor module estimates the current device state S^t as per which the proposed IDS execution scheme switches its primary objective.

Algorithm 2 presents the proposed solution. Its inputs are the energies consumed by the device during packet transmissions in cycle t extracted as a vector ε^t . Its output is

the current estimated system state S^t . The *Schedule* function (shown in lines 1 to 15) is executed throughout the lifetime of the network. The aging and the backoff counters (m and p , respectively) are initialized to 1 each (shown in line 2). For cycle $t = 0$, the thresholds for both the total packet transmission energy, E_{th}^0 , and that for the confidence score, C_{th}^0 , are initialized to 0.5 each (shown in line 3). The HIDS is executed for cycle $t = 0$ to obtain the initial confidence score as C^0 (shown in line 4). Here, the *Update* function, discussed later, is called to generate a device state and update the thresholds and the counter variables (shown in line 5). From here, all the next steps are repeated for each cycle $t > 0$. The change in traffic load for cycle t is obtained as E^t (shown in line 7). The backoff space is obtained from the backoff module as $b^t = p^3$ (shown in line 8), and the aging scheme is obtained from the aging module as $a^t = \frac{m^2}{b^t}$ (shown in line 9). The confidence score is updated using the aging scheme (shown in line 10). The device trusts d^t is computed (shown in line 11), and a weighted random choice r^t between 0 and 1 with d^t favoring 0 is obtained (shown in line 12). The decision to execute the IDS is taken if at least one of the two conditions mentioned in section III-F is met (shown in line 13). If the decision is negative, the aging counter m is incremented by 1 (shown in line 15). Otherwise, the confidence score C^t obtained from the IDS execution (shown in Line 14) is used to call the *Update* function (shown in line 16).

The *Update* function (shown in lines 17 to 28) generates a system state for the current cycle t . It also updates the thresholds and the counter variables for the next cycle. It accepts as inputs the change in traffic load E^t , the updated confidence score C^t , the threshold E_{th}^t for change in traffic load, and the threshold C_{th}^t for change in confidence score for cycle t . If the confidence score is $C^t \geq 1$, the device state S^t is set to *Attack* (shown in line 19). As per the dynamic threshold generation scheme, the thresholds for changes in traffic load and confidence score for the next cycle are updated as the average of the current cycle thresholds and half of the current cycle changes (shown in lines 20 to 23). The aging counter m is also reset to 1 (shown in line 24). If the confidence score $C^t < 1$, the device state is set to *Benign* (shown in line 26), and the backoff counter p is incremented by 1 (shown in line 27). If $p > 10$, then it is reset to 1 (shown in line 28). Three implementations of the proposed framework were tested. The complete framework (Algorithm 2) as *Combined*, a confidence-based scheme (omit lines 7, 20 and 22 from Algorithm 2) as *Confidence*, and a traffic based scheme (omit lines 2, 8, 9, 15, 21, 23, 24, 27 and 28 from Algorithm 2) as *Traffic*, with appropriate update of the execution condition in line 13.

IV. PERFORMANCE ANALYSIS

Simulations have been carried out using the Cooja simulator in Contiki-NG [25] to generate data. The radio model was chosen as UDGM with RPL Lite as the routing protocol. All host devices were created using the z1 platform and were set to the transmission range of 50 meters. A total number of host

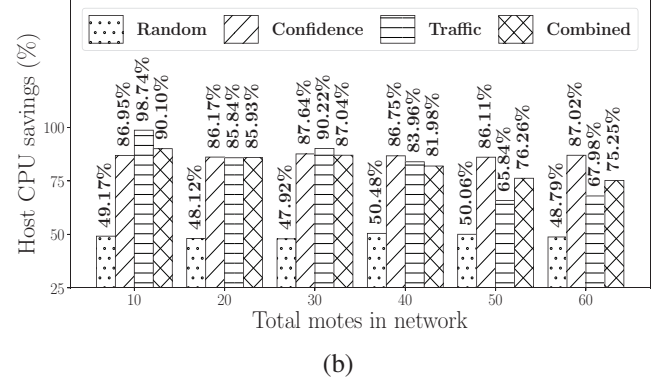
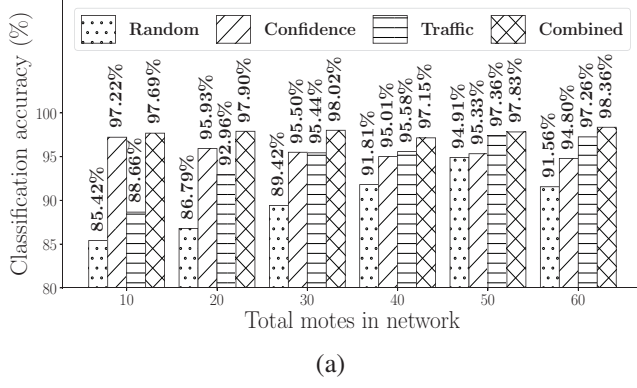


Fig. 2. Results related to the primary scheduling objectives under different device states for the scheduling policies: Random, Confidence, Traffic, and Combined. (a) Device behavior classification accuracy during the attack. (b) Device CPU saving when not under attack.

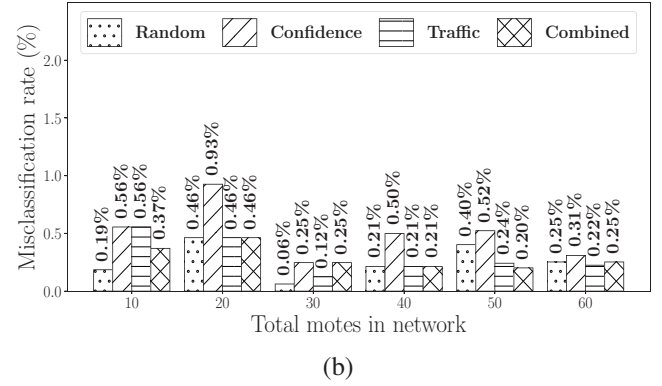
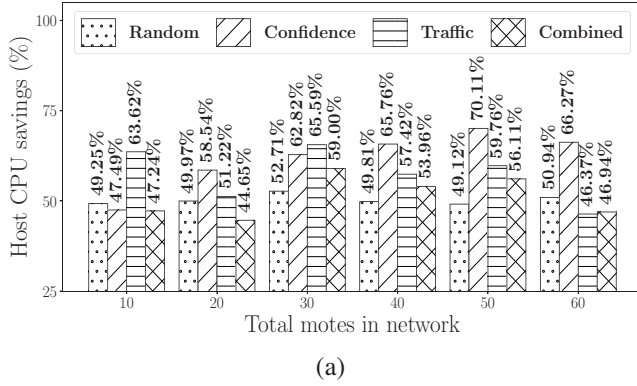


Fig. 3. Results related to the secondary scheduling objectives under different device states for the scheduling policies: Random, Confidence, Traffic, and Combined. (a) Device CPU saving during attack. (b) Device behavior misclassification rate when not under attack.

devices varied between 10 – 60 with 10% attack devices, and all were distributed randomly. Simulations were carried out for 8 hours with the last 6 hours as attack duration. A predefined example in Contiki-NG was executed in which all client devices send predefined packets aperiodically to the root node using a multihop topology. The Energest module in Contiki-NG was used to periodically capture host device behavior in terms of around 14400 entries of CPU computation and packet transmission energies of each device and saved to text files. Each device had a modified firmware to incorporate the HMA attack programmatically initiated by simulated button clicks at 2 hours of simulation start. The HIDS and the proposed framework were implemented using Python 3. The HIDS was trained using an AMD Ryzen 7 Pro 5250G CPU having 8 physical cores and 16 GB RAM running Ubuntu 22.04. The training was done without any GPU support for 200 epochs with a batch size of 256, a learning rate of 0.001, and 32 timesteps. The attack probability tolerance $P_{th} = 80\%$ and cycle duration $|t| = 200$ seconds were used in the simulations.

Fig. 2 and Fig. 3 present the simulation results related to the primary and secondary scheduling objectives under different device states obtained for three implementations of the proposed framework, namely, confidence-based (*Confidence*), traffic load based (*Traffic*) and the combination of both (*Combined*).

For comparison with a state-of-the-art that detects HMA, the random-based *WindRain* IDS scheduling (*Random*) [15] was chosen and fit to IoT network environment. All implementations have been compared against the baseline of the HIDS executed without scheduling. Fig. 2(a) and Fig. 2(b) present the performances of the schemes in terms of device behavior classification accuracy during an attack and CPU saving during a benign device state, respectively. Fig. 2(a) shows that the *Combined* scheme outperforms all the schemes in device behaviour classification at 97.15% for 40 devices in the network. Fig. 2(b) shows that the CPU savings of all the implementations of the proposed scheme are comparable, at 81.98%, while that of the state-of-the-art is the lowest, at 50.48%, for the same network. This shows that the proposed *Combined* scheme outperforms all other schemes in the primary objective of high HMA detection accuracy during an attack and high CPU savings during a benign device state.

Fig. 3(a) and Fig. 3(b) present the performances of the schemes in terms of CPU saving during an attack and behavior misclassification rate during the benign device state, respectively. Fig. 3(a) shows that the CPU saving of the proposed *Combined* scheme during device attack, at 53.96%,

is outperformed by the other implementations of the proposed scheme, while it is comparable with that of the state-of-the-art, at 49.81%. Fig. 3(b) shows that all the schemes are comparable in their behavior misclassification rate during benign device state, at 0.21%. Thus, the *Combined* scheme performs par with the other schemes in the secondary objective of high CPU saving during an attack and low device behavior misclassification rate when not under attack. Therefore, the proposed scheduling framework adapts to the device state to maximize its dynamic objectives accordingly. Thus, it achieves better HMA detection accuracy with comparable CPU saving during attacks and better energy savings with comparable classification accuracy in benign scenarios than those of the other schemes.

V. CONCLUSION

It is observed that IDS solutions that can detect HMAs have a portion of their implementation executing in the host devices and consume device resources at each execution. Such existing solutions mostly have random-based scheduling or do not use scheduling at all. Random-based scheduling solutions have low HMA detection accuracy, while solutions that do not use scheduling do not save device resources. To address these issues, an IDS scheduling framework has been proposed. To schedule the IDS for high resource saving without much degradation in HMA detection accuracy due to scheduling, the proposed framework uses a monitor module for runtime device state estimation, a threshold generator for dynamic threshold modification, a backoff scheme for trust-based IDS execution suspension, an aging scheme for revival of the suspended IDS, and an execution scheme for device state-based objective switching for IDS scheduling. Extensive simulations confirm that the proposed framework achieves a device behavior classification accuracy of at least 97.15% when under attack while saving at least 81.98% of CPU energy when not under attack for a network with 40 devices, as compared with those when executing the IDS without a schedule. It is to be noted that the energy saving of the proposed framework, though comparable with the other schemes, is not the best among them. In the future, work will be done to improve the energy saving of the host devices further while retaining the HMA detection accuracy of the proposed framework.

REFERENCES

- [1] I. A. Khan, D. Pi, M. Z. Abbas, U. Zia, Y. Hussain, and H. Soliman, "Federated-SRUs: A federated simple recurrent units-based IDS for accurate detection of cyber attacks against IoT-augmented industrial control systems," *IEEE Internet of Things Journal*, vol. 10, no. 10, pp. 8467–8476, 2022.
- [2] C. Parampalli, R. Sekar, and R. Johnson, "A practical mimicry attack against powerful system-call monitors," in *Proc. of the ACM symposium on Information, computer and communications security*, 2008, pp. 156–167.
- [3] V. Kumar and O. P. Sangwan, "Signature based intrusion detection system using SNORT," *International Journal of Computer Applications & Information Technology*, vol. 1, no. 3, pp. 35–41, 2012.
- [4] M. Al-Fawa'reh, J. Abu-Khalaf, P. Szewczyk, and J. J. Kang, "MalBoT-DRL: Malware botnet detection using deep reinforcement learning in IoT networks," *IEEE Internet of Things Journal*, vol. 11, no. 6, pp. 9610–9629, 2024.
- [5] S. Bhadauria and T. Mohanty, "Hybrid Intrusion Detection System using an Unsupervised method for Anomaly-based Detection," in *Proc. of IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS)*, 2021, pp. 1–6.
- [6] U. Sabeel, S. S. Heydari, K. El-Khatib, and K. Elgazzar, "Unknown, Atypical and Polymorphic Network Intrusion Detection: A Systematic Survey," *IEEE Transactions on Network and Service Management*, vol. 21, no. 1, pp. 1190–1212, 2023.
- [7] S. Ahn, H. Yi, Y. Lee, W. R. Ha, G. Kim, and Y. Paek, "Hawkware: network intrusion detection based on behavior analysis with ANNs on an IoT device," in *Proc. of 57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6.
- [8] B. T. Beasley, G. D. O'Mahony, S. G. Quintana, A. Temko, and E. Popovici, "Lightweight Anomaly Detection Framework for IoT," in *Proc. of 31st Irish Signals and Systems Conference (ISSC)*, 2020, pp. 1–6.
- [9] H. Kim, S. Ahn, W. R. Ha, H. Kang, D. S. Kim, H. K. Kim, and Y. Paek, "Panop: Mimicry-Resistant ANN-Based Distributed NIDS for IoT Networks," *IEEE Access*, vol. 9, pp. 111 853–111 864, 2021.
- [10] Z. Lv, H. Guo, J. Hu, Z. Zhang, and Z. Wu, "A Binary Bee Foraging Algorithm Based Feature Selection Approach for IoT Intrusion Detection," *IEEE Internet of Things Journal*, vol. 11, no. 5, pp. 7604–7618, 2024.
- [11] A. Ghubaish, Z. Yang, A. Erbad, and R. Jain, "LEMMA: A Novel Feature Engineering Method for Intrusion Detection in IoT Systems," *IEEE Internet of Things Journal*, vol. 11, no. 8, pp. 13 247–13 256, 2024.
- [12] M. Abdel-Basset, H. Hawash, R. K. Chakraborty, and M. J. Ryan, "Semi-supervised spatiotemporal deep learning for intrusions detection in IoT networks," *IEEE Internet of Things Journal*, vol. 8, no. 15, pp. 12 251–12 265, 2021.
- [13] Y.-C. Lai, D. Sudyana, Y.-D. Lin, M. Verkerken, L. D'hooge, T. Wauters, B. Volckaert, and F. De Turck, "Task assignment and capacity allocation for ML-based intrusion detection as a service in a multi-tier architecture," *IEEE Transactions on Network and Service Management*, vol. 20, no. 1, pp. 672–683, 2022.
- [14] E. Nowroozi, M. Mohammadi, E. Savaş, Y. Mekdad, and M. Conti, "Employing deep ensemble learning for improving the security of computer networks against adversarial attacks," *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 2096–2105, 2023.
- [15] S. P. Chung and A. K. Mok, "On random-inspection-based intrusion detection," in *Recent Advances in Intrusion Detection (RAID)*. Springer, 2006, pp. 165–184.
- [16] P. Bhale, D. R. Chowdhury, S. Biswas, and S. Nandi, "OPTIMIST: lightweight and transparent IDS with optimum placement strategy to mitigate mixed-rate DDoS attacks in IoT networks," *IEEE Internet of Things Journal*, vol. 10, no. 10, pp. 8357–8370, 2023.
- [17] A. Le, J. Loo, K. K. Chai, and M. Aiash, "A specification-based IDS for detecting attacks on RPL-based network topology," *Information*, vol. 7, no. 2, pp. 25–44, 2016.
- [18] T. Halabi, O. A. Wahab, R. Al Mallah, and M. Zulkernine, "Protecting the Internet of vehicles against advanced persistent threats: A bayesian Stackelberg game," *IEEE Transactions on Reliability*, vol. 70, no. 3, pp. 970–985, 2021.
- [19] K. Hammar and R. Stadler, "Learning near-optimal intrusion responses against dynamic attackers," *IEEE Transactions on Network and Service Management*, vol. 21, no. 1, pp. 1158–1177, 2023.
- [20] W. Wang, Y. Shang, Y. He, Y. Li, and J. Liu, "BotMark: Automated botnet detection with hybrid analysis of flow-based and graph-based traffic behaviors," *Information Sciences*, vol. 511, pp. 284–296, 2020.
- [21] J. E. Tapiador and J. A. Clark, "Masquerade mimicry attack detection: A randomised approach," *Computers & Security*, vol. 30, no. 5, pp. 297–310, 2011.
- [22] R. DiPietro and G. D. Hager, "Deep learning: RNNs and LSTM," in *Handbook of medical image computing and computer assisted intervention*. Elsevier, 2020, pp. 503–519.
- [23] M. Riedmiller and A. Lermen, "Multi layer perceptron," *Machine Learning Lab Special Lecture, University of Freiburg*, vol. 24, 2014.
- [24] W. H. L. Pinaya, S. Vieira, R. Garcia-Dias, and A. Mechelli, "Autoencoders," in *Machine learning*. Elsevier, 2020, pp. 193–208.
- [25] G. Oikonomou, S. Duquenois, A. Elsts, J. Eriksson, Y. Tanaka, and N. Tsiftes, "The Contiki-NG open source operating system for next generation IoT devices," *SoftwareX*, vol. 18, pp. 101 089–101 097, 2022.